

SOFTWARE ENGINEERING

PROJECT PROPOSAL AND SOFTWARE REQUIREMENTS SPECIFICATION

Voice Assistant for Multimedia Controls

Group No.: 12

Team Members:

1. Prajwal GT | SRN: PES2UG25CS818
2. Shreedhar R | SRN: PES2UG25CS823
3. Shrinivas S J | SRN: PES2UG24CS494
4. Sarang S Nair | SRN: PES2UG24CS447

PART I – PROJECT PROPOSAL

1. Project Title

Voice Assistant for Multimedia Controls

2. Team Profile

Sl. No.	Name	SRN
1	Prajwal GT	PES2UG25CS818
2	Shreedhar R	PES2UG25CS823
3	Shrinivas S J	PES2UG24CS494
4	Sarang S Nair	PES2UG24CS447

3. Proposed Project Description

The proposed project is a hardware-based Voice Assistant for Multimedia Controls. It is designed to provide hands-free control of common multimedia operations through simple voice commands. The system will use a Raspberry Pi or ESP-based controller, a microphone, and Python-based software to capture, process and interpret predefined voice commands. The recognized command will then be converted into the corresponding multimedia action on a connected computer or media device.

The core version will support volume up, volume down, play, resume, pause, next and previous operations. MediaPipe may also be integrated to recognize predefined hand gestures and map them to selected multimedia controls. This provides an additional hands-free interaction method when speaking is inconvenient.

The project is proposed as a new software-hardware integrated system. It is intended for students and general users who frequently use music, video or presentation applications and want an alternative to keyboard, mouse or touchscreen controls. The system will be developed as a modular prototype so that additional commands and control methods can be added later.

4. Target Users

- Students using computers for music, videos and presentations.
- Users who require or prefer hands-free multimedia control.
- Users who want to control media from a short distance.
- Users who want an alternative to traditional keyboard and mouse controls.

5. Functional Features

- Recognize predefined voice commands.
- Increase media volume using a voice command.
- Decrease media volume using a voice command.
- Play or resume currently paused media.
- Pause currently playing media.
- Move to the next media track or video.
- Move to the previous media track or video.
- Provide feedback when a command is recognized.
- Ignore unsupported or unrecognized commands safely.
- Support selected gesture-based controls using MediaPipe, if implemented.

6. Plan of Work

Week	Planned Work
1	Finalize requirements; identify hardware/software requirements; study voice recognition and multimedia control methods.
2	Set up Raspberry Pi/ESP, microphone and computer communication; develop basic voice input module.
3	Implement play, pause, resume, volume up and volume down commands.
4	Implement next and previous controls and integrate multimedia commands.
5	Implement selected MediaPipe gesture recognition and gesture-to-command mapping.
6	Integrate modules; perform testing; fix errors; improve response time and recognition accuracy.
7	System-level testing, final demonstration, documentation and feedback.

7. Product Ownership and Team Responsibilities

Team Member	Functional Features	Qualitative Contribution
Prajwal GT PES2UG25CS818	Voice command detection; play/resume; pause	Improve voice-command recognition accuracy and reduce response delay.
Shreedhar R PES2UG25CS823	Volume up; volume down; Raspberry Pi/ESP and device communication	Ensure reliable hardware-to-device communication and consistent volume control.
Shrinivas S J PES2UG24CS494	Next; previous; command feedback	Ensure reliable track navigation and clear feedback when commands are recognized.
Sarang S Nair PES2UG24CS447	MediaPipe gesture recognition; gesture-to-command mapping; system integration	Improve gesture recognition accuracy and reduce unintended gesture triggers.

Note: All team members are expected to participate in every major project activity, while the above responsibilities identify each member's primary ownership of specific functional features.

8. Quality Goals

- Performance: recognized commands should be executed with minimal delay.
- Accuracy: supported commands should be recognized correctly under normal conditions.
- Reliability: the system should operate without frequent crashes or interruptions.
- Usability: commands and gestures should be simple and easy to understand.
- Maintainability: modules should be structured so new commands can be added easily.
- Portability: the prototype should run on suitable Raspberry Pi or ESP-based hardware.
- Robustness: invalid commands, background noise and temporary communication failures should be handled gracefully.
- Privacy: voice input should not be unnecessarily stored or transmitted externally.

9. Expected Outcome

The expected outcome is a working prototype that allows users to control common multimedia operations through voice commands, with optional gesture-based control using MediaPipe. The project demonstrates the application of software engineering principles to a practical hardware-software system involving voice recognition, computer vision, hardware interfacing and multimedia control.

PART II – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements of the Voice Assistant for Multimedia Controls. It serves as a common reference for development, testing and evaluation of the proposed system.

1.2 Scope

The system will capture voice input through a microphone, identify supported commands, and perform corresponding multimedia actions through a hardware controller and connected computer/media device. Optional MediaPipe-based gesture recognition will provide an additional control mechanism.

1.3 Definitions

Term	Meaning
Voice Command	A predefined spoken instruction used to control media.
Multimedia Control	An operation such as play, pause, volume or track navigation.
Raspberry Pi/ESP	Hardware controller used to interface input and control functions.
MediaPipe	A framework that can be used for computer-vision and hand-gesture recognition.

2. Overall Description

2.1 Product Perspective

The product is an integrated hardware-software prototype. A microphone provides voice input, the processing layer identifies the command, and a control layer sends the appropriate multimedia action to the target device or media player.

2.2 Product Functions

- Voice input acquisition and command recognition.
- Play/resume and pause control.
- Volume increase and decrease.
- Next and previous media navigation.
- Optional hand-gesture recognition.
- Command feedback and error handling.

2.3 User Characteristics

The intended users are general computer and multimedia users. No specialized technical knowledge should be required to issue the predefined commands.

2.4 Assumptions and Dependencies

- A suitable microphone and supported Raspberry Pi/ESP hardware are available.
- The target computer or media device accepts the required multimedia control actions.
- Voice recognition software/libraries are available for the selected implementation.

- MediaPipe functionality depends on availability of a compatible camera and processing environment.
- The system is primarily designed for predefined commands rather than unrestricted natural-language conversation.

3. External Interface Requirements

3.1 Hardware Interfaces

- Microphone for capturing voice commands.
- Raspberry Pi or ESP-based controller for system interfacing.
- Optional camera for MediaPipe gesture recognition.
- Network or wired/wireless connection where required for communication with the target device.

3.2 Software Interfaces

- Python-based processing and control software.
- Voice recognition/input processing library or service selected during implementation.
- Media control interface or operating-system multimedia commands.
- MediaPipe, if gesture recognition is implemented.

3.3 User Interface

The primary user interface is voice-based. The user speaks a predefined command and the system performs the corresponding action. Optional visual or audio feedback may indicate whether a command was recognized.

4. Functional Requirements

ID	Requirement
FR-01	The system shall capture voice input through a microphone.
FR-02	The system shall identify supported predefined voice commands.
FR-03	The system shall increase media volume when the user issues the volume-up command.
FR-04	The system shall decrease media volume when the user issues the volume-down command.
FR-05	The system shall pause currently playing media when the pause command is recognized.
FR-06	The system shall play or resume media when the play/resume command is recognized.
FR-07	The system shall move to the next media item when the next command is recognized.
FR-08	The system shall move to the previous media item when the previous command is recognized.
FR-09	The system shall provide feedback after a supported command is recognized.
FR-10	The system shall safely ignore unsupported or unrecognized commands.
FR-11	The system shall optionally recognize predefined hand gestures using MediaPipe.
FR-12	The system shall map configured gestures to corresponding multimedia actions when gesture control is enabled.

5. Non-Functional Requirements

ID	Requirement
NFR-01	Performance: the system should respond to a valid command with minimal noticeable delay.
NFR-02	Accuracy: supported voice commands should be recognized correctly under normal operating conditions.
NFR-03	Reliability: the system should operate continuously without frequent crashes.
NFR-04	Usability: commands should be simple, consistent and easy to remember.
NFR-05	Maintainability: voice recognition, gesture recognition and media control should be modular.
NFR-06	Portability: the system should support the selected Raspberry Pi/ESP platform.
NFR-07	Low Power: the hardware should be suitable for continuous, low-power operation.
NFR-08	Security: only intended commands should trigger supported actions.
NFR-09	Privacy: voice data should not be unnecessarily stored or transmitted.
NFR-10	Robustness: background noise, invalid commands and temporary connection problems should be handled gracefully.
NFR-11	Compatibility: the system should work with the selected supported multimedia player/device.

6. Use Cases

Use Case	Actor	Precondition	Main Action	Expected Result
Play/Resume	User	Media player is available	User says play/resume	Media starts or resumes.
Pause	User	Media is playing	User says pause	Media pauses.
Volume Up	User	Media device is connected	User says volume up	Media volume increases.
Volume Down	User	Media device is connected	User says volume down	Media volume decreases.
Next	User	Media playlist has a next item	User says next	Next item starts.
Previous	User	Media playlist has a previous item	User says previous	Previous item starts or is selected.
Gesture Control	User	Gesture mode is enabled	User performs configured gesture	Mapped multimedia action is executed.

7. System Workflow

- System starts and initializes the microphone/controller.
- System waits for voice input or a configured gesture.
- Input is captured and processed.
- The system compares the input with supported commands/gestures.

- If a valid command is identified, the corresponding multimedia action is executed.
- The system provides feedback to the user.
- If the input is invalid or unsupported, no unintended media action is performed.
- The system returns to the listening/ready state.

8. Testing Requirements

Test Area	Example Test
Voice Recognition	Speak each supported command and verify correct identification.
Playback	Verify play, resume and pause actions.
Volume	Verify volume increases and decreases correctly.
Navigation	Verify next and previous commands.
Invalid Input	Verify unsupported speech does not trigger an action.
Noise Handling	Test commands under normal background noise.
Gesture Control	Verify configured gestures map to the correct actions.
Integration	Verify hardware, Python software and media player work together.
Reliability	Run the system continuously and check for crashes or unintended actions.

9. Future Enhancements

- Support for additional multimedia applications and devices.
- Customizable voice commands.
- More gesture-based controls.
- Wake-word activation.
- Support for multiple languages.
- Improved offline voice recognition.
- Integration with smart-home or IoT devices.

10. Conclusion

The Voice Assistant for Multimedia Controls is a practical software engineering project that combines voice interaction, embedded hardware and multimedia control. The SRS defines a clear baseline for implementation and testing, while the proposal establishes the project scope, work plan and product ownership responsibilities for the team.